



A Computation-In-Memory Accelerator Based on Resistive Devices

Hoang Anh Du Nguyen, Jintao Yu, Muath Abu Lebdeh, Mottaqiallah Taouil, Said Hamdioui

Delft University of Technology

Delft, The Netherlands

{H.A.DuNguyen},{J.Yu-1},{M.F.M.AbuLebdeh},{M.Taouil},{S.Hamdioui}@tudelft.nl

ABSTRACT

Today's computing architectures suffer from the three well-known bottlenecks, which are the memory, the power and the instruction-level parallelism walls. Emerging non-volatile technologies, such as memristor, enable new resistive architectures that alleviate at least two of such bottlenecks, as they can process data within the memory with almost no leakage. In this paper, we propose a novel resistive computing architecture by extending a conventional architecture with a resistive based Computation-In-Memory accelerator (CIMX). We evaluate the delay, energy and area of the conventional and CIMX architecture using an analytical model and a simulation framework. The results (both based on the analytical model and simulation framework) show that the proposed architecture achieves at least one order of magnitude improvement in terms of performance, area, and energy efficiency for the considered benchmarks.

KEYWORDS

Computation-in-Memory, Accelerator, Resistive Computing, Memristor

ACM Reference Format:

Hoang Anh Du Nguyen, Jintao Yu, Muath Abu Lebdeh, Mottaqiallah Taouil, Said Hamdioui. 2019. A Computation-In-Memory Accelerator Based on Resistive Devices. In *Proceedings of the International Symposium on Memory Systems (MEMSYS '19)*, September 30-October 3, 2019, Washington, DC, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3357526.3357554>

1 INTRODUCTION

Performance improvement of current computing architectures is gradually saturating due to three well-known bottlenecks: memory wall, power wall and instruction-level parallelism (ILP) wall [1]. These bottlenecks are direct consequences of the von Neumann load/store architecture, speed gap between processor and memory, and the difficulty of exploiting the maximum parallelism, respectively. In order to address these walls, novel architectures using emerging technologies are under research as an alternative for future computing systems [2].

Memristor is one of the promising technologies to enable new architectures [3, 4, 5, 6] due to its great scalability, high integration

density, and its near-zero standby power, etc. [2, 7, 8]. Based on such devices, several potential resistive computing architectures (e.g., Computation-In-Memory (CIM) [3, 9, 10], Resistive Associate Processor [11, 5], Programmable Logic-in-Memory (PLiM) [4], Pinatubo [6] etc.) have been proposed as an alternative to today's load/store architectures. The preliminary results of these resistive architectures show several orders of magnitude improvement for different metrics such as energy and area efficiency [3, 9]. However, most of the proposed memristor logic designs (such as threshold/majority logic [12, 13], implication logic [14, 15] and Boolean logic [16, 17, 18]) to build such architectures require multiple accesses of the memristor devices in order to perform a single logic operation; these do not only impact the latency, but more important, they significantly reduce the lifetime of the devices (due to limited endurance) and increases the overall write energy. In [6, 19], the authors partially addressed these problems by performing logic operations, while reading the memory, but then using customized read circuitry (e.g., sense amplifiers). During these read operations, the memristors do not switch and hence, the device lifetime is not affected and the overall energy is reduced. Nevertheless, the authors require multiple steps to perform XOR operations. More importantly, as the nonvolatile memory is deployed as the main memory, it does not entirely solve the endurance problem due to frequent write operations. In addition, the authors did not consider communication infrastructure between the processor and memory (i.e., maintaining memory coherency when parts of the computation occur in memory).

In this work, we propose a novel Computation-in-Memory (CIM) core based on scouting logic. In our previous work [20], we explored the possibility of performing logic operations within non-volatile memories by sensing (and *scouting*) the data stored in the memory, similarly as in [6]. Hence, a memory access provides by itself the result of a pre-defined logic operation. 'Scouting' the current or voltage drop while accessing two (or more) operands at the same time can be used to perform a logic function (e.g., by using a modified sense amplifier). Different from [6], the non-volatile CIM memory is used as an accelerator with infrequent writes (thereby addressing the endurance) and moreover, scouting logic is able to execute all logic operations within a single cycle (thereby addressing the performance) rather than multiple cycles. The contributions of this paper are the following: (1) a novel computation-in-memory core that utilizes scouting logic; we refer to this accelerator as CIM core; (2) an extension of the conventional instruction set with in-memory instructions that supports code execution of kernels on CIM core; (3) an analytical evaluation model of the conventional and CIM-based architectures; it can be used for fast design space exploration; (4) a simulation framework that is able to evaluate both

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MEMSYS '19, September 30-October 3, 2019, Washington, DC, USA

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-7206-0/19/09...\$15.00

<https://doi.org/10.1145/3357526.3357554>

the conventional architecture and CIM-based architecture. Three kernels of different applications are used as case studies.

The remainder of this paper is organized as follows. Section 2 presents the state-of-the-art in memristor logic design, the working principle of scouting logic, and its potential. Section 3 presents potential resistive architectures and the CIM accelerator. Section 4 provides an analytical evaluation model, and design exploration of the conventional and proposed architecture. Section 5 presents a simulation framework and evaluates several benchmarks. Section 6 subsequently evaluates the results, discusses the potential of CIM-based architecture based on the analytical and simulation results. Finally, Section 7 concludes this paper.

2 SCOUTING LOGIC AND ITS POTENTIALS

This section first describes the state-of-the-art in memristor based logic design. Thereafter, it explains the working principle of scouting logic. Finally, it discusses its performance, pros and cons.

2.1 State-of-the-art on Memristor-Based Logic

Several memristor-based logic designs have been proposed [13, 12, 14, 15, 16, 18, 17, 21]. The most popular design suitable for computation-in-memory are Snider logic [18, 16], implication logic [15, 14] and Memristor-aided (MAGIC) logic [22]. They are briefly discussed next.

Snider logic provides two primitive gates: NOT and NAND [16]. It uses high resistance R_H and low resistance R_L to represent a logic 1 and 0, respectively. The two-input NAND gate of Fig. 1(a) is used as an example to explain the working principle of Snider logic. We assume that the inputs are stored in memristor M_1 and M_2 , while the output is produced in M_0 . The gate requires an extra resistor R_s ($R_L \ll R_s \ll R_H$). To perform NAND, two steps are needed. First, M_0 should be RESET to R_H . Next, control voltages V_h (half-select voltage) and V_w (write voltage) are applied to the input and output memristors, respectively; V_h and V_w need to satisfy the following constraint: $V_w > V_{reset} > V_h = \frac{V_w}{2}$, where V_{reset} is the minimum voltage to switch a memristor from R_L to R_H [23, 18]. In case one of the inputs is 0, the equivalent resistance of M_1 and M_2 is around R_L (see e.g. Fig. 1(a) where $M_1=R_L$ and $M_2=R_H$). Therefore, the voltage V_x across the resistor R_s is around V_h as $R_L \ll R_s \ll R_H$. As a result, the voltage across the output memristor M_0 is $V_w - V_x \approx V_w - V_h = \frac{V_w}{2} < V_{reset}$, and therefore M_0 stays at R_H (logic 1).

Implication logic provides only a single primitive gate, which is material implication (IMP) [15]. Opposite to Snider logic, IMP uses R_H and R_L to represent logic 0 and 1, respectively. IMP gate uses two memristors M_1 and M_2 to store the inputs; the output is overwritten into memristor M_2 . The working principle of IMP is similar to Snider logic. Multiple sequential IMP gates can be used to realize AND, OR, or XOR gates.

MAGIC logic provides two primitive gates: NOR and NOT [22], where the NOT gate is constructed in a similar way as the NOR gate but without the second input. Similar to implication logic, MAGIC uses R_H and R_L to represent logic 0 and 1, respectively. Fig. 1(b) shows a NOR gate; memristors M_1 and M_2 store the inputs, while the output is produced in M_0 . To perform NOR, two steps are needed. First, M_0 is SET to R_L . Next, control voltages V_w (write voltage)

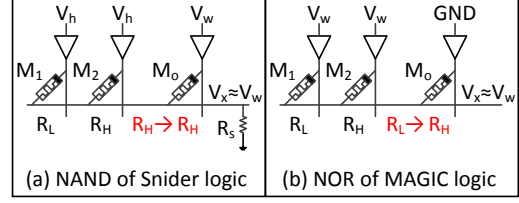


Figure 1: Logic designs suitable for resistive computing

and GND (Ground) are applied to the input and output memristors, respectively. Note that the value of V_w must satisfy the following constraints: $2V_{reset} < V_w < V_{set}$, where V_{set} is the minimum voltage required to switch a memristor from R_H to R_L . In case one of the inputs are 1, the equivalent resistance of M_1 and M_2 is around R_L . Therefore, the voltage across M_0 is $V_x \approx \frac{V_w}{2} > V_{reset}$ and the output memristor switches to R_H (logic 0).

The three previous discussed designs suffer from major concerns. First, the limited memristor endurance puts a constraint on using these designs, as they require multiple write steps/accesses to execute a single operation. In addition, the multiple write steps also affect the performance. Second, processing a logic operation requires several control voltages (e.g., V_w) that have to drive memristor devices, therefore, higher power requirement. All of these indicate the severe limitation of such designs in realizing computation-in-memory architectures. One alternative design to solve these limitations is to perform the operation within the peripheral circuits of the memory, as will show next.

2.2 Scouting Logic

Scouting logic executes logic operations by modifying the read circuitry [20]. Fig. 2(a) shows a resistive memory with two cells based on 1T1R cells. Normally when a cell is read, (e.g., memristor M_1), a read voltage V_r is applied to its row and switch S_1 is activated. Subsequently, a current I_{in} will flow through the bit line to the input of the sense amplifier (SA). This current is compared to the reference current I_{ref} . If I_{in} is greater than I_{ref} (i.e., when R_1 has a low resistance R_L), the output of the SA changes to V_{dd} (logic 1). Similarly, $I_{in} < I_{ref}$ (i.e., when R_1 has a high resistance R_H), the output changes to logic 0. For proper operations, I_{ref} should be fixed between the high and low current as depicted in the top left part of Fig. 2(b).

Inspired by this read operation, we demonstrate how to implement OR, AND and XOR scouting logic gates, which are frequently used bitwise logic operations [24, 25]. Instead of reading a single memristor at a time, scouting logic activates two (or more) inputs of the gate simultaneously (e.g., M_1 and M_2 in Fig. 2(a)). As a result, the input current to the sense amplifier is determined by the equivalent input resistance ($R_1 // R_2$). This resistance results in three possible values: $\frac{R_L}{2}$, $\frac{R_H}{2}$ and $(R_L // R_H) \approx R_L$. Hence, the input current I_{in} also can have only three values. By changing the value of I_{ref} , different gates can be realized. To implement an OR gate, I_{ref} should be set between $\frac{2V_r}{R_H}$ and $\frac{V_r}{R_L}$ as depicted in the left bottom part of Fig. 2(b)). As a result, the output is 0 only when $R_1 // R_2 = \frac{R_H}{2}$. Similarly, to implement an AND operation, I_{ref} should be set between $\frac{2V_r}{R_L}$ and $\frac{V_r}{R_H}$. The XOR operation needs two references and the output is logic 1 only when $R_1 // R_2 \approx R_L$.

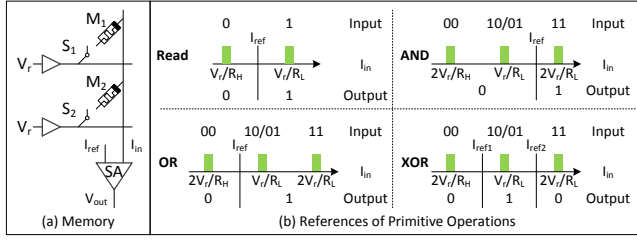


Figure 2: Scouting logic [20]

Table 1: Parameters

Parameter	Description	Value
Technology		
Memristor (TaO _x) [27, 2]		
F (nm)	Feature size	90
R_L (k Ω)	Low resistance	200
R_H (M Ω)	High resistance	10
A_{cell} (μm^2)	Area of a 1T1R cell	0.0486
T_{sw} (ns)	Switching time (max of SET and RESET)	1.71
V_r (V)	Read voltage	0.9
CMOS		
UMC 90nm Library		

Table 2: Different operation performance

	Delay (ns)	Power (μW)			Area (μm^2)
		OR	AND	XOR	
CSA	2.73	15.99	17.19	17.59	29.7216
VCA	9.31	8.65	6.00	11.01	22.324

The above implies two requirements for the SA. First, it should be operational with both a single and two references. Second, it must support a re-configurable reference. Two SA designs including a current-based SA (CSA) and voltage-based SA (VSA) that satisfy both requirements are described in [20]; in addition, scouting logic is verified using Cadence Spectre simulations. The simulation model consists of the 1T1R array of Fig. 2(a) connected to one of the SA designs; both SAs have been verified. The 1T1R array and SAs are described by a SPICE netlist, while the memristor model [26], CMOS controller and voltage drivers by Verilog-A modules. The simulation parameters are extracted from [27] and summarized in Table 1. The SA designs are simulated with the PTM 90nm [28] library. The functionality of scouting logic has been verified, and the delay, power and area of the OR, AND and XOR operation for both SA designs (including the cost of their controllers) have been reported; see Table 2 [20]. The CSA design is much faster (3.4x), but has a higher area overhead (1.5x) and consumes more power (1.6-2.8x). In this paper, we select VSA design for the following section due to its favourably lower power consumption.

As a relatively old technology is used, the read voltages mentioned are typically a bit higher than reported [7]. To predict the performance for a 5nm memristor [2] design, we scaled these numbers down. It is worth mentioning that lowering the read voltage might impact the correctness of the scouting logic, as it is strongly dependent on the development of both memristor technology and the modified sense amplifier design.

2.3 Potentials of Scouting Logic

Compared to the state-of-the-art resistive logic designs, scouting logic executes the logic operations during a read operation. Therefore, scouting logic comes with the following advantages:

- Scouting logic executes a logic operation within a single access (i.e., read operation), and hence it improves the performance.
- Scouting logic requires lower control voltages (i.e., V_r is lower than the write voltage V_w), and hence it reduces the power and energy consumption together with better performance.
- Scouting logic only reads the resistance values of memristors instead of writing them. As a result, it has a lower endurance requirement, which is currently a significant challenge for memristor technology [29].
- Scouting logic requires simple control logic, as it does not need to control the sequential steps required by other resistive logic designs, such as implication logic.

3 RESISTIVE COMPUTING ARCHITECTURE

In this section, we first describe potential Computation-in-Memory (CIM) architectures that can utilize scouting logic. Thereafter, we select and explain the most suitable one together with its instruction set in more detail.

3.1 Potential Architectures

With scouting logic, it is feasible to implement a non-volatile memory that supports CIM. In other words, the memristor-based memory module is equipped with one or more scouting logic units. Fig. 3b, 3c, 3d, and 3e show four possible ways to embed a CIM core into the conventional computer architecture of Fig. 3a; they are respectively CIM used as cache, as main memory, as complete memory hierarchy, and as accelerator. Next, we discuss the pros and cons of each of these architectures.

Fig. 3b shows the CIM cache (CIMC) architecture where CIM replaces one or more cache levels. Here, CIM operates in a similar way as conventional caches by storing temporary data that will likely be reused. However, besides the normal cache functionality, CIM can be used to perform in-memory operations. With respect to the normal cache functionality, no modifications are required to the processor, as these internal cache functionality is transparent to processor. The in-memory operations are performed in the cache and prevent moving data towards processors. Hence, it reduces the cost (i.e., latency and power) of moving data between caches and processors, and vice-versa. In addition, CIM provides larger cache size due to small footprint of resistive devices [30, 31, 32]. However, as parts of the instructions are executed in caches, the processors must be made aware of these operations. Therefore, some modifications are required for both the cache controller and processor instruction set to handle coherency. They impact the instruction set, compiler and cache controller. Moreover, as cache is more frequently accessed than main memory, the limited endurance and high read/write latency of resistive devices are the main bottlenecks of this approach [31]. Hence, this architecture makes sense only if the endurance is high enough ($>10^{16}$).

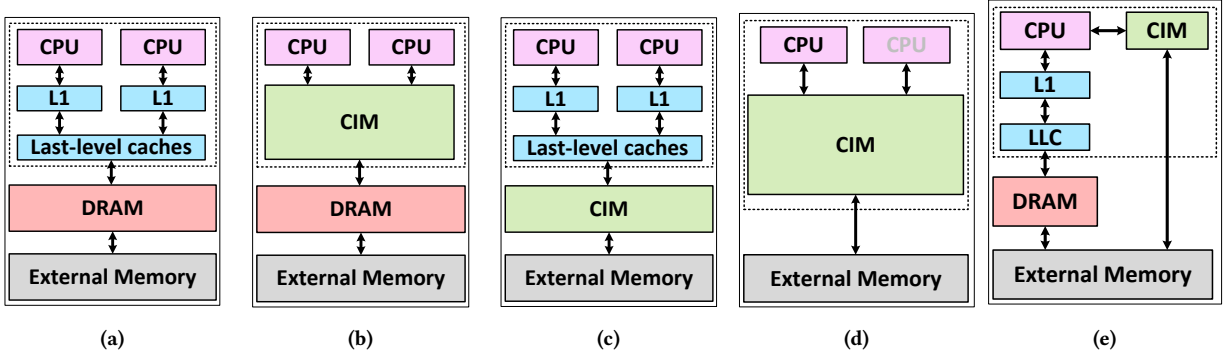


Figure 3: Conventional (a) and potential resistive architectures: CIMC (b), CIMM (c), CIMU (d), CIMX (e)

Fig. 3c shows the “CIM main” (CIMM) where CIM replaces the main memory. Here, CIM operates similarly as a conventional DRAM memory. The scouting logic operations can be used in addition to normal read operations to perform bit operations in the memory; a similar approach has been evaluated for DRAM in [33]. The trade-off between capacity and performance is similar as in the approach where CIM is used to replace the cache. In fact, resistive devices in general are often used as non-volatile memory devices. Several prototypes of up to 32GB memory have been reported recently [34, 35, 36]. These prototypes show a lot of advantages in using resistive devices in main memory such as high density, non-volatility, nano scale devices [37]; these advantages enable high storage volume and low start-up time [37, 38, 39, 40]. However, a lot of efforts are still required to use non-volatile memories in conventional architectures due to some challenges such as high manufacturing cost, high write latency, endurance limitation, and memory coherence management and complexity [41, 40, 42].

Fig. 3d shows the CIM universal (CIMU) where CIM replaces the entire memory hierarchy. One or multiple processors are used to execute the conventional instructions and fetch data directly from CIM; in-memory instructions can be still used to accelerate memory-intensive parts of an application. This reduces the pressure on the registers. However, to be competitive in terms of performance, the memory must have a low write and read latency. In addition, as a lot of write operations are expected also the endurance needs to be high. Despite possible performance issues, this architecture could have a huge potential in terms of energy and area efficiency.

Fig. 3e shows the CIM accelerator (CIMX) where CIM is used as an on-chip data-centric accelerator. CIM executes parts of the instructions that cannot be efficiently handled by conventional processors; for example, a loop that consists of simple operations on a huge data set. As accelerators are often read-favored, the impact on endurance is expected to be minimal. The data-centric accelerator differs from a traditional accelerator such as an FPGA or GPU in the amount of data that can be stored in the accelerator; this huge data storage alleviates the latency and energy cost required to move data back and forth from memory in traditional accelerators. However, it also has commonalities with the traditional accelerators. First, the performance gain is proportionally dependent on the number of parallel operations that can be performed simultaneously. Second, the memory coherence management is managed in a similar way, i.e., the accelerator can be seen from the processor as an extended

memory space, similarly as for FPGA accelerators [43, 44]. Third, the accelerators can be executed in parallel with the host processor in case no data dependencies occur.

All the four discussed CIM-based architectures have both pros and cons. In this paper, we evaluate the resistive architecture depicted in Fig. 3e, as it is the only case where CIM can be used as an accelerator and thus has a lower requirement on the endurance. Currently, an endurance of 10^6 to 10^{12} are reported [7, 29]; this is neither sufficient for general purpose computing nor conventional memory. However, it is strongly believed that memristor endurance will increase and reach 10^{16} in the future [29]. In the next sub-section, we will present the CIMX architecture in detail.

3.2 Resistive Computing Architecture

The CIMX architecture of Fig. 3e consists of a conventional processor with cache, main DRAM memory, novel data-centric CIM core and an external memory. CIMX consists of a large non-volatile memory equipped with scouting logic. Both main memory and CIM core can fetch data from the external memory. CIM core is addressable from the processor and uses an extended address space. In this paper, we assume that the data that is stored in the CIM core is not duplicated on the main DRAM memory [45, 46]; hence, no memory coherence schemes are required. The CIM core is initialized with data from the external memory, e.g., database(s); this initialization needs to be performed only once. Note that infrequent modifications are required when read-dominant applications are executed on CIM core.

Similarly as for the FPGA based accelerators [43, 44], pragma insertions are used to denote that particular functions will be executed on CIM core. The pragma insertions are translated into a sequence of in-memory instructions; these are executed in CIM core. The instruction format for an in-memory instruction is constructed in a similar manner as for conventional processor instructions. It includes an opcode to denote the in-memory instruction type and the addresses to operate on. To signal that this operation should not be executed on the conventional processor, a specific opcode is used for the instruction. As a result, the instruction is directly forwarded to the memory controller for further processing.

Fig. 4 presents the internal details of CIM core. The accelerator consists of a memory controller, configuration registers, address decoder, non-volatile memory cell arrays, and a read/write circuitry. The *controller* receives in-memory instructions from the processor;

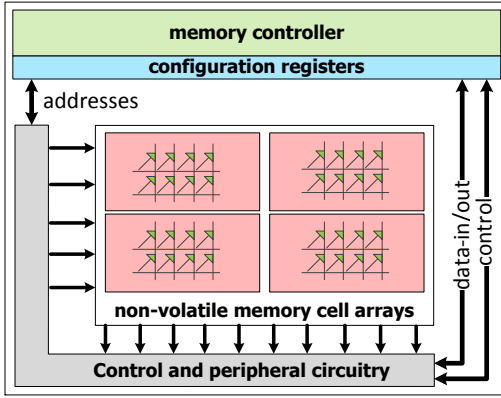


Figure 4: CIM core

each instruction specifies the operation type (read, write, logic or, etc.) and addresses to operate on. The configuration registers are used to decode the instructions and configure the data path. Once the operation finishes, the final result is sent back to the processor in case applicable. The *control and peripheral circuitry* specify which rows are activated and performs a read (including a logical operation) or write operation. The *non-volatile memory* consists of multiple arrays where each array consists of a 1T1R crossbar array (i.e., each cell consists of a transistor and memristor).

It is worth noting that the CIM core considered here can only execute logical operations. More research is required to extend the instruction set with arithmetic operations. Due to this current limitation, the next section will only consider logical operations in CIM core.

3.3 CIM Instruction Set Architecture

In order to support the execution of instructions in CIM core, we create an CIM instruction set based on scouting logic operations. The CIM instruction set includes two types of instructions related to memory and processing. Memory related instructions are typical load/store instructions used to retrieve/save data from/into memory. Processing related instructions consist of logical operations that are executed by scouting logic as mentioned in section 2.

Here we consider the logic instructions CIM_AND, CIM_OR, CIM_XOR representing the and, or, and xor instructions as explained in section 2, respectively. We add an additional compare instruction denoted by CIM_CMP; this instruction will be needed for one of the benchmarks that will be simulated in section 5.2 (i.e., bitmap indexing [33, 47]). The CIM_CMP is composed of a CIM_XOR instruction followed by a bit-wise OR operation; the later operation is executed within the CMOS peripheral circuitry. The instruction set can be easily extended with arithmetic operations when their implementations are available.

4 ANALYTICAL EVALUATION RESULTS

In this section, we first present an analytical evaluation model for both the conventional and CIMX architecture. Thereafter, we evaluate the performance of both architectures using these models. Finally, we describe their shortcomings and limitations.

4.1 Analytical Evaluation Model

In order to evaluate CIMX, we would like to compare its performance, energy consumption and area with a conventional architecture. To be able to realize this, we create two models one for the conventional and one for CIMX architecture. The models can be used to evaluate different applications.

We characterize an application based on its problem size and percentage of logical instructions. The problem size specifies the amount of program instructions. The percentage of logical instructions is the amount of accelerated instructions (X) that can be offloaded to CIM core. We denote the percentage of logical instructions by n_l , remaining instructions by n_r (arithmetic, conditional, etc.) and the percentage of their corresponding memory accesses by m_l , and m_r , respectively. Note that $n_l + n_r = 1$.

For the **conventional architecture**, we use the Intel Xeon E5-2680 [48] multicore as a baseline. We assume the following:

- The number of cores is $n_p=4$, each with a frequency of 2.5GHz.
- Each core contains (i) an ALU that is capable of executing non-memory instructions (logical, arithmetic, conditional instructions) in one cycle, (ii) a two level cache (L1 of 32KB and L2 of 256KB) with access latencies of one and two cycles [49], and a miss rate of mr_{L1} and mr_{L2} , respectively. Note that the actual miss rates are application dependent.
- The cores share a main DRAM memory of 4GB with an access latency of 175 cycles (165 cycles for communication and 10 cycles for retrieving the results) [49]. The page fault rate of the main DRAM memory is $pf_{dr}=0.1\%$ with a penalty of 800 cycles [50].

For the **CIMX architecture**, we assume the following:

- It contains a single host processor with the same characteristics as an individual core in the conventional architecture. It contains the following:
 - (i) an ALU that executes non-logical instructions in one cycle.
 - (ii) 32KB L1 cache and 256KB L2 cache. As parts of the application are offloaded to CIM core, the miss rates of the part of the application that are executed on CPU changes. To model this change, we use the *square root* rule of miss rate [51, 52]. This rule says how the cache miss rate changes for different cache sizes. However, in our models we keep the cache sizes between both architecture the same. Instead, we assume that the same relation holds for different problem sizes (due to offloading parts of the application to CIM core). Note that when the problem sizes decreases, it has a similar effect of increasing the cache. Specifically, the miss rates for CIMX (due to accelerated parts) are modeled by $(1 - X)^{(-0.3)}$ in which X represents the accelerated ratio and -0.3 the minimum coefficient in the *square root* rule, i.e., the most pessimistic value.
- 1GB DRAM.
- a CIM core with $n_a=1,048,576$ parallel memory arrays (each with a size of 512 columns x 512 rows) with a total capacity equal to 32GB and an area of nearly equal to 3GB DRAM.

- Unidirectional communication between the processor and CIM core is assumed to be equal to L1 access latency (i.e., 1 cycle). Note that both L1, L2 cache and CIM core are on-chip.
- A logical instruction takes 9.3 ns on CIM core based on the VSA (see Table 2); this is equivalent to 20 CPU cycles.

Defining $m = m_l + m_r$ and $m_p = \frac{m}{n_p}$, the CPI of both architectures is modeled as below:

$$CPI_{CONV} = \begin{cases} \frac{(n_l + n_r) \cdot 1}{n_p} + \dots & t_{comp} \\ m_p \cdot (1 - mr_{l1}) + \dots & t_{L1} \\ m_p \cdot mr_{l1} \cdot (1 - mr_{l2}) \cdot 2 + \dots & t_{L2} \\ m \cdot mr_{l1} \cdot mr_{l2} \cdot (1 - pf_{dr}) \cdot 175 + \dots & t_{DRAM-hit} \\ m \cdot mr_{l1} \cdot mr_{l2} \cdot pf_{dr} \cdot 800 & t_{DRAM-pf} \end{cases} \quad (1)$$

$$CPI_{CIMX} = \begin{cases} n_r \cdot 1 + \dots & t_{comp-others} \\ \frac{n_l}{n_a} \cdot 4 + \dots & t_{comp-bw} \\ 1 + \dots & t_{CPU-CIM} \\ \frac{n_l}{n_a} \cdot 4 & t_{CIM-write} \\ m_r \cdot (1 - mr_{l1}) + \dots & t_{L1} \\ m_r \cdot mr_{l1} \cdot (1 - mr_{l2}) \cdot 2 + \dots & t_{L2} \\ m_r \cdot mr_{l1} \cdot mr_{l2} \cdot 175 & t_{DRAM-hit} \end{cases} \quad (2)$$

The CPI of the conventional architecture (see Eq. 1) consists of the sum of the CPU computing latency t_{comp} , the L1 cache accessing latency t_{L1} , the L2 cache accessing latency t_{L2} , the DRAM memory accessing latency assuming a main memory hit $t_{DRAM-hit}$, and the DRAM page fault accessing latency $t_{DRAM-pf}$. Note that for the conventional architecture, there is no differentiation between logical and non-logical instructions and their corresponding memory accesses (i.e., $m=m_l+m_r$).

The CPI model of CIMX (see Eq. 2) consists of the sum of the non-logical instruction latency $t_{comp-others}$ executed by the processor, the logical instruction compute latency $t_{comp-bw}$ executed by the CIM core, the on-chip communication latency between CPU and CIM core $t_{CPU-CIM}$ estimated by 1 cycle, the non-volatile write latency $t_{CIM-write}$, the L1 cache accessing latency t_{L1} , the L2 cache accessing latency t_{L2} , the DRAM memory accessing latency for a hit rate $t_{DRAM-hit}$. Note that it is assumed that CIM core is large enough to store all the data needed for the accelerator.

Table 3 summarizes the technology parameters. The table contains the delay, area, and power consumption of the conventional CPU (i.e., the logic operations, caches (L1, L2) and DRAM) and of CIM core (i.e., crossbar array and scouting logic). Note that the scouting logic related parameters were already provided in Table 2. Here, we assume the VSA implementation due to its favourably lower power consumption. The data provided in Table 3 is used to evaluate the delay, energy, and area of each architecture. The delay is calculated from the CPI models and the problem size. The energy is computed from the delay, and dynamic and static power. The area is obtained by summing up the areas of all individual components. Note that the controllers of both the conventional and CIMX architectures are not included in the estimation.

For the evaluation, we use the following metrics:

- Performance: defined by the delay (in s).

Table 3: Evaluation parameters

Component	Parameters	
CMOS Technology		
32-bit logic gate	Delay (cycles)	1
	Dynamic power (uW)	33.69 [53, 54, 55]
	Static power (uW)	7.09
	Area (um ²)	32.65 [53, 54]
32KB L1 cache	Delay (cycles)	1
	Dynamic power (W)	0.24 [56]
	Static power (W)	0.012 [56]
	Area (mm ²)	14.2 [56]
256KB L2 cache	Delay (cycles)	2
	Dynamic power (W)	0.63 [56]
	Static power (W)	0.108 [56]
	Area (mm ²)	75 [56]
1GB DRAM	Delay (cycles)	175
	Dynamic power (W)	12.86 [56]
	Static power (W)	12 [56]
	Area (mm ²)	88.98 [56]
4GB DRAM	Delay (cycles)	175
	Dynamic power (W)	51.4 [56]
	Static power (W)	48 [56]
	Area (mm ²)	355 [56]
Memristor Technology		
Scouting logic operation	Delay (cycles)	4
	Power (W)	see Table 2
	Area (mm ²)	
Memristor	Write Delay (cycles)	1
	Write energy (fJ)	1 [2]
	Static energy (J)	0 [2]
	Area (nm ²)	6F ² =150 [2]
Non volatile memory	#col_per_row	512
	#row_per_array	512
	#array	1,048,576

- Energy: defined by energy (in J).
- Performance energy efficiency: defined by the number of million operations per second divided by the power consumption (in MOPS/mW).
- Energy efficiency: defined by the energy in pJ divided by the number of operations (in pJ/op).
- Performance area efficiency: defined by the number of million operations per second divided by the total area (in MOPS/mm²).

4.2 Analytical Results

Based on the models developed in the previous section, we will evaluate the performance of the conventional and CIMX architectures for the 32GB problem size (PS); as this equals the CIM core capacity, we assume that initial data related to the accelerated part is stored in CIM core and does not required to be reloaded. We investigate the impact of two aspects on the performance of both architectures: (i) the L1 and L2 cache miss rates (ranging from 0% to 90%), and (ii) the percentage of logical instructions accelerated by CIM core %Acc. (including 30%, 60%, and 90% as the authors in [6] showed that at least 30% of a database application could be accelerated using in-memory computing). Using this analytical evaluation, we can quickly perform a design space exploration. Fig. 5 shows the performance metric of the conventional architecture

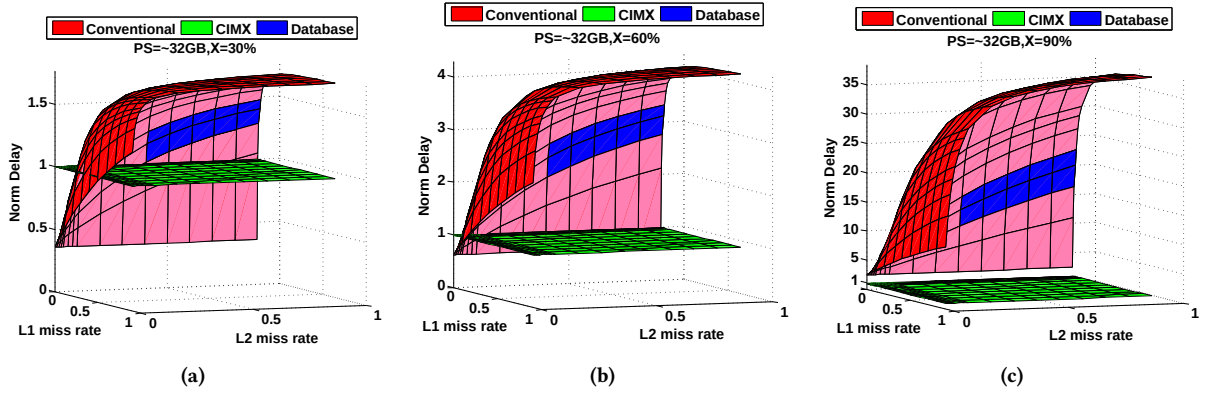


Figure 5: Analytical results of the performance (delay) metric for the conventional and CIMX architectures

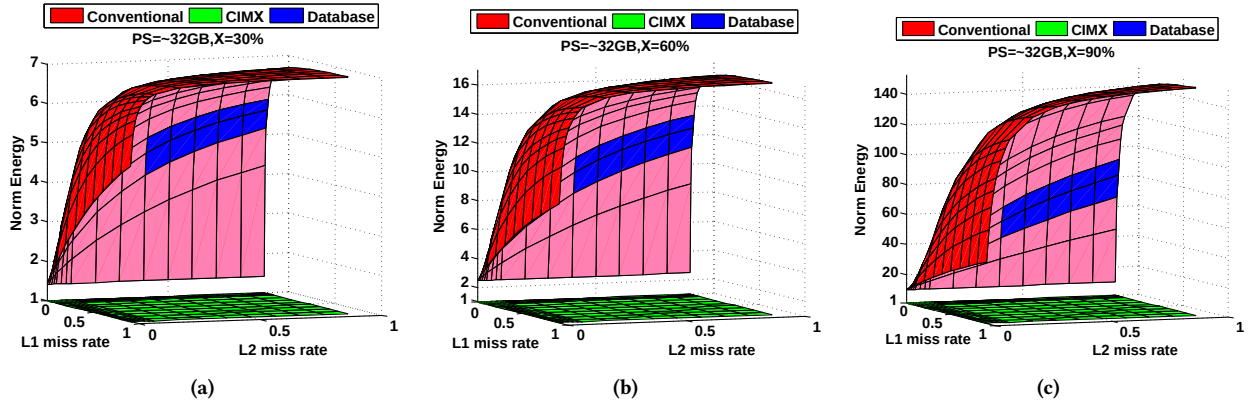


Figure 6: Analytical results of the energy metric for the conventional and CIMX architectures

(red planes) with respect to CIMX architecture (green planes). The results of the conventional architecture are normalized to those of the CIMX; note that the results of the CIMX architecture are presented by the green unit planes. The blue areas on the red planes are explained later. Overall, the performance speed-up of CIMX architecture increases for larger percentages of the instructions executed on CIM core (maximum 1.5x for 30% to more than 35x for 90%). This can be clearly observed as the gap between the red and green planes increases. CIMX architecture outperforms the conventional architecture for most of the L1 and L2 miss rates. High cache miss rates result in a longer memory access latency in the conventional architecture while CIMX architecture does not suffer much from this as computing takes place within the memory (i.e., CIM core). However, CIMX architecture may not achieve a speed-up for low miss rates (e.g., less than 20%). For example, when the miss rates of L1 and L2 are around 10%, a specific amount of instructions (e.g., more than 60%) have to be accelerated to realize a higher performance on CIMX architecture. Note that the miss rate of some big data applications is reported around 6–15% [57, 58, 59]. However, most big data applications such as database application have a much higher miss rates; this will be discussed further later in this sub-section. Moreover, the figure shows that the performance increase saturates for higher L1 and L2 miss rates. The reason for this is that the program executed on the conventional part of CIMX

starts to dominate and that therefore the accelerated percentage is bounded by Amdahl's law.

Similarly, Fig. 6 shows the energy metrics for both architectures. Overall, similar trends are observed with respect to the percentage of accelerated instructions. However, the energy consumption of CIMX architecture is always lower, irrespective of the cache miss rates. In case 30% of the instructions are accelerated on CIM core, the conventional architecture consumes 1.5x to 7x less energy. This grows up to 140x in case 90% of the instructions are accelerated on CIM core. The high energy consumption of the conventional architecture can be partly attributed to the data movement back and forth between the CPU and the memory hierarchy. In addition, both cache and DRAM suffer from a much higher leakage current as compared with the non-volatile technology used in CIM core.

Fig. 7 shows the analytic results of the remaining metrics for the 90% acceleration ratio case; note that the results of CIMX are normalized to those of the conventional architecture in the performance energy efficiency and performance area efficiency. CIMX architecture achieves an improvement of two orders of magnitude in terms of performance energy efficiency, energy efficiency, and one order of magnitude in terms of performance area efficiency, mostly due to a higher performance (see Fig. 5c) and lower energy consumption (see Fig. 6c). To the best of our knowledge, CIMX architecture is the first architecture that integrates a conventional

architecture with a scouting logic based accelerator. In comparison to the state-of-the-art, the proposed architecture realizes significant improvements, despite the low performance of individual memristors. The improvement could be much higher if more instructions can be accelerated, and/or if the CIM core performance can be improved. Furthermore, applications with a bad data locality and high data volume can be significantly benefit from the proposed architecture.

We also explore database applications which have an L1 and L2 miss rate ranging from 2 to 4%, and 40 to 90%, respectively [57]. The performance of CIMX in these ranges are marked in blue in Figs. 5, 6, and 7. For these particular miss rates, database applications can be executed up to 15x faster on CIMX architecture depending on the accelerated percentages. Meanwhile, CIMX architecture consumes 5x to 60x less energy than conventional architecture. This results in one order of magnitude improvement in terms of performance energy efficiency, energy efficiency, and performance area efficiency. This exploration quickly shows that database applications are a suitable candidate to be accelerated on CIMX architecture.

The proposed architecture also shows more than 150x improvement in energy efficiency. Therefore, it can be considered as a potential platform for low-power or portable embedded applications. To make CIMX architecture better suitable for low power/portable embedded applications, the complex memory hierarchy has to be removed (i.e., no L2 cache and DRAM). We evaluated the impact of the L1 cache miss rate (ranging from 0% to 70%) and percentage of accelerated instructions (ranging from 30% to 90%) for the gigabyte (GB) problem size. Fig. 8 shows the evaluation metrics of such modified architecture. We observe that the modified architecture achieves a performance energy efficiency higher than 100 MOPs/mW, which is much better than most application-specific designs [60]. In addition, the energy efficiency is reaching the requirement of 1pJ/op for embedded applications [60] as shown in Fig. 8(b). This shows the necessity of integrating resistive devices in embedded systems to realize energy and area efficiency.

4.3 Limitations of Analytical Model

The analytical model assumes a uniform profile of the application (e.g., the same L1 miss rate throughout the entire application). However, an application consists normally of multiple parts with different behavior, such as memory-intensive, computation-intensive or bandwidth-intensive parts. In addition, modeling the communication between the kernel is not trivial. Furthermore, both architectures do not consider the cost of the memory controller due to their modeling complexity. For these reasons, a simulation framework is necessary. Nevertheless, the analytical approach can be used for quick insights and to perform a fast exploration. In the next section, we propose a simulation framework that can be used to simulate both conventional architectures (Fig. 3a) and CIMX architecture (Fig. 3e).

5 SIMULATION FRAMEWORK AND RESULTS

In this section, we first explain the simulation framework. Thereafter, we discuss the simulation setup and applied benchmarks. Finally, we present the simulation results for both the conventional and CIMX architectures.

5.1 Simulation Framework

Fig. 9 shows the simulation framework (SiCIM) used to simulate the two architectures. The inputs of the framework consist of three components: application, architecture and technology parameters. The application is a program or benchmark (written in C/C++ language) possibly annotated to specify the parts that must be executed on the CIM core. The architecture parameters specify the simulated architecture such as instruction sets, cache size, cycles per instruction, etc. The technology parameters contain technology related info such as technology node, frequency and power consumption. Although there is a strong dependency between the technology and architectural parameters, this dependency is not shown in the simulation framework. Instead, we have simulated this dependency for the kernels outside this framework using HSpice by involving both the technological and architectural parameters. The outputs of the framework consist of the delay, energy and area of the simulated application.

The core of the framework consists of three phases as shown in Fig. 9, and explained next. Note that in order to support CIMX simulation, multiple existing tools are used and possibly modified, such as Intel Pin [61], MCPProf [62], SiNUCA [63], mCPAT [64], NVSIM [65], Cacti [66].

Pre-processing phase: in the pre-processing phase, a profiling tool (i.e., MCPProf [62]) is used to extract the memory-intensive parts of the application. Code 1 illustrates an example where the memory intensive part consists of a "for loop". To force the execution of this part in CIM core, pragma's are added (as of now manually) to the code as shown in Code 2 (lines 6 and 10). After profiling, SiNUCA trace generator [63], which uses Intel Pin [61], generates three types of traces: (i) static traces containing all instructions that have to be executed (ALU, memory, branch instructions), (ii) dynamic traces containing information related to the control flow such as branch instructions (i.e., taken or not taken), (iii) memory traces containing the memory addresses of read and write instructions. Note that the use of pragma's for CIM core instructions will result in updated traces. To realize this, we have extended the trace generator with the new CIM instructions, similarly as performed in [67]. For example, the OR operation of line 8 in Code 1 is replaced by an OR operation on CIM core (`_cim_btw_or`) in Code 2; the `_cim_btw_or` instruction is defined in the library of the trace generator and used to generate the updated traces for CIMX, similarly as performed for HMC instructions in [67].

Processing phase: in the processing phase, the SiCIM engine simulates the generated traces by executing them on the specified architecture (i.e. the processor type, cache size, DRAM size, CIM core configuration, etc.). For CIMX architecture, the non-accelerated part of the program is executed on the host processor, while the CIM core instructions are dispatched from the host processor to the CIM core. The SiCIM engine is based on the SiNUCA simulator [63] which we extended with CIM instructions. The SiCIM engine generates execution statistics of the simulation, which includes the number of cycles to execute the program, the number of memory accesses (cache and main memory), and the number of CIM instructions. The CIM core instructions are executed under a lock/unlock scheme to maintain data coherency similarly as in HIVE [63, 68]; this scheme ensures that data-dependent instructions are executed

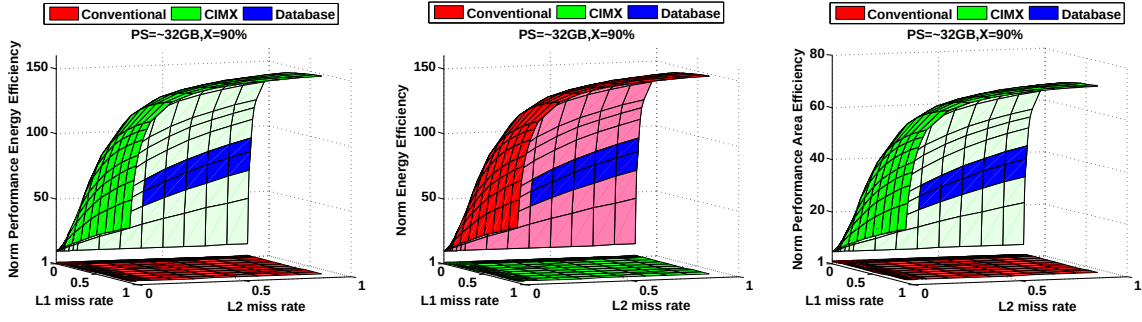


Figure 7: Analytical results of the performance energy, energy, and performance area efficiency metrics

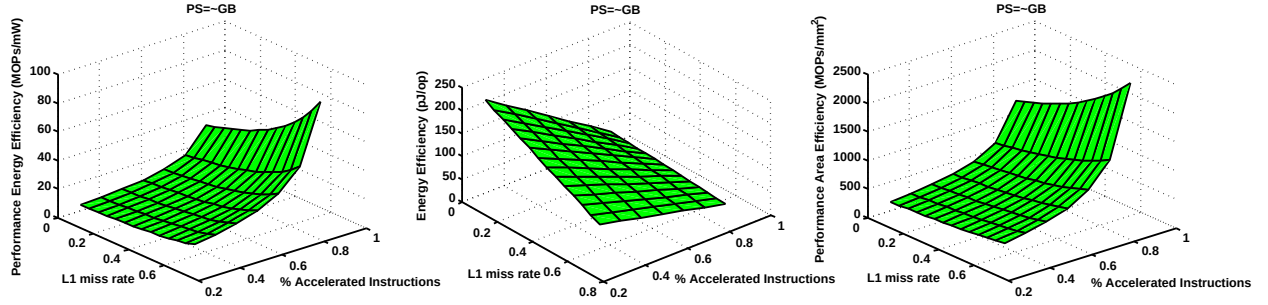


Figure 8: Analytical results for modified CIMX architecture

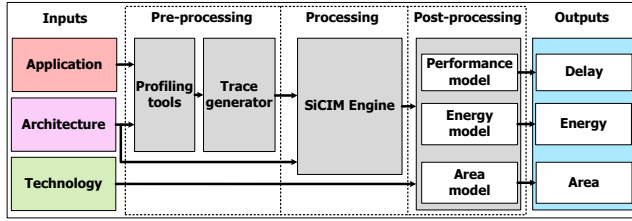


Figure 9: Simulation framework

```

1  ...
2  int main(int argc, char* argv[]) {
3      ...
4      for (size_t i=0; i<1000; i++) {
5          c[i] = a[i] | b[i];
6      }
7      ...
8  }

```

Code 1: Source Code for Conventional Architecture

in the correct order, while data-independent instructions can be executed in parallel between the host and accelerator.

Post-processing phase: in the post-processing phase, the output statistics of the processing phase are used to calculate the delay, energy and area metrics using a combination of three tools: (i) mcPAT, a processor simulator [64], (ii) NVSIM, a non-volatile memory simulator [65], and (iii) Cacti, a cache and main memory (DRAM) simulator [66].

```

1  ...
2  #include " ../hmc.hpp"
3  ...
4  int main(int argc, char* argv[]) {
5      ...
6      MCPROF_START();
7      for (size_t i=0; i<1000; i++) {
8          c[i] = _cim_btw_or(&a[i], b[i]);
9      }
10     MCPROF_STOP();
11     ...
12 }

```

Code 2: Source Code for CIMX Architecture

5.2 Simulation Setup and Benchmarks

The simulation parameters of both architectures are summarized in Table 4. The conventional architecture is based on an Intel Xeon CPU E5-2650 that consists of a single 64-bit core, 32KB L1, 256KB L2 and 1GB DRAM. CIMX architecture has a similar build up, but instead has 500MB DRAM memory and a CIM core with a capacity of 500MB; note that data is assumed to be initialized in the DRAM and CIM core; hence, the cost of data initialization is not considered in the simulation time as it is not the focus of this work. This configuration is selected to maintain a reasonable simulation time and capability (i.e., Cacti, NVSIM, SiNUCA) for both architectures. The processor operates in both architectures on 64-bit data widths, while CIM core operates on 1024-Byte data widths, which is equivalent to the execution of $1024^8/64=128$ CPU instructions in parallel.

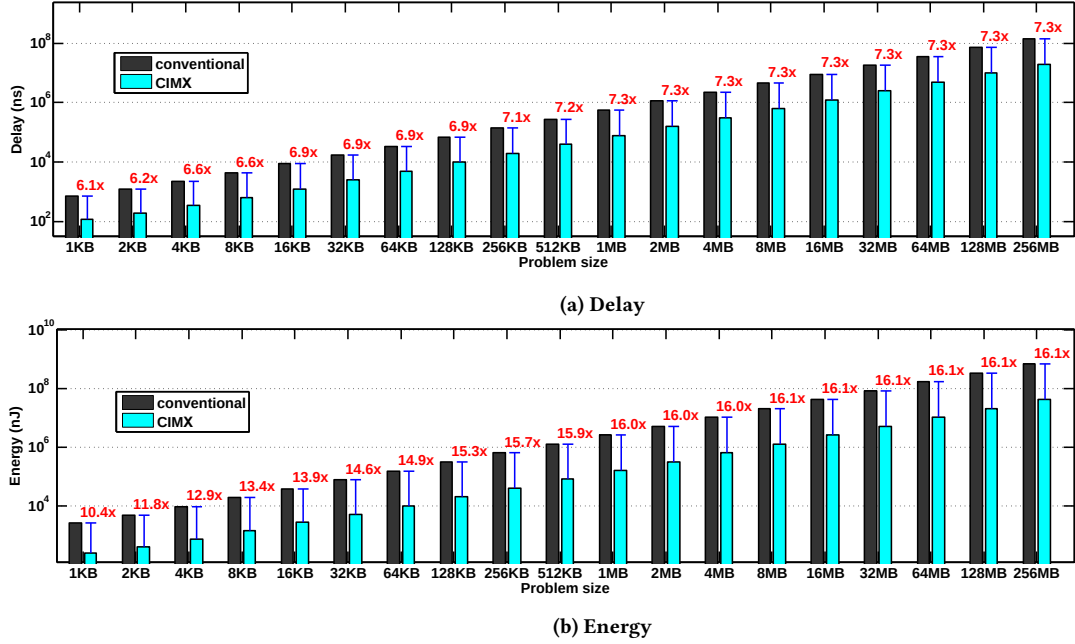


Figure 10: Performance comparison for vector OR kernel

Table 4: Architecture configuration

Configuration	Conventional Arch.	CIMX Arch.
Processor	1 core @ 2GHz	1 core @ 2GHz
Caches	32KB L1	32KB L1
	256KB L2	256KB L2
Main memory	1GB DRAM	500MB DRAM
		500MB CIM

In this paper, we simulate three frequently-used application kernels on both the conventional and CIMX architecture, without modifying the underlying algorithms. They are the *vector OR*, *XOR encryption*, and *QUERY SELECT* database kernel. They are the memory-intensive parts of vector processing [69], encryption [70] and database applications [71], respectively. They contain bulk bitwise operations that perform poorly on conventional architectures due to high cache miss rates [58, 72, 57, 73]. The three kernels are further described next.

- The *vector OR* kernel performs an OR operation of two vectors. The size of the vectors represents the simulated problem size. In case the vector size is larger than the data-width of a single OR instruction (64-bit for CPU and 1024-byte for CIMX), multiple OR instructions are executed using full loop unrolling.
- The *XOR encryption* kernel performs an XOR operation of a string sequence and a predefined (secret) key. It is used for one-time-pad cryptography [70]. The size of the string sequence represents the problem size. In case the string sequence size is larger than the data-width of a single XOR

instruction (64-bit for CPU and 1024-byte for CIMX), multiple XOR instructions are executed using a loop with multiple iterations.

- The *QUERY SELECT* database kernel performs the query-06 of the TPC-H benchmark [74], which includes 22 queries written in SQL language. In order to simulate these queries, the queries need to be converted to C/C++ language (to be fed into the simulation framework). As this process is not trivial, the manually created trace for query-06 selection with a 1GB database size presented in [75] is used. The query-06 performs compare instructions (as mentioned in Section 3.3) and checks if the requested data is available in the database or not; this includes conjunctions and disjunctions in large volume tables without join operations between tables. Note that a complete query execution may require other operations (e.g., min, max, etc.) that must be performed on the host processor side. However, in this work, we only focus on the kernel that performs bitwise operations.

5.3 Simulation Results

Fig. 10 shows the simulation results of the delay and energy consumption of both architectures for the vector OR kernel, respectively. The results clearly show that CIMX architecture outperforms the conventional architecture by 6x and 7x in terms of delay, and by 10x and 16x in terms of energy consumption. The benefits are mainly due to computation-in-memory (reduction in data transfer) and use of non-volatile technology. Moreover, the obtained delay and energy improvements strongly depend on the amount of supported parallelism; note that the more instructions can be executed in parallel, the higher the advantage of using CIMX. For example, with a larger problem size, the delay speed-up increases from 6.1x to 7.3x.

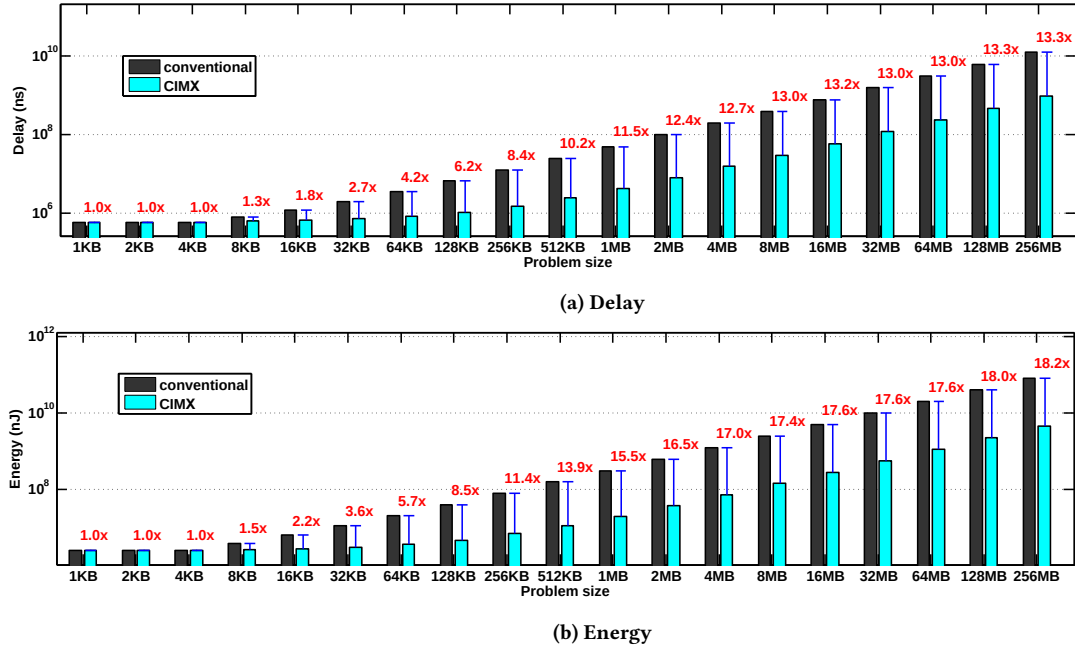


Figure 11: Performance comparison for XOR encryption kernel

Similarly, Fig. 11 shows the results of the XOR encryption kernel. For small problem sizes (less than 4KB), CIMX has no delay and energy improvement due to the overhead of non-accelerated part. However, the improvement is nearly 2x when the problem size is doubled. The XOR encryption is more complex than the vector OR kernel, as the non-accelerated part (i.e. the part that executes on the host CPU) consists of a much larger fraction of the total execution code. In case of large problem sizes, (i.e. larger than 512KB), CIMX architecture achieves more than one order of magnitude improvement both in terms of delay and energy in comparison with the conventional architecture. The speed-up results from two factors: (i) the exploited parallelism similarly as in the case of vector OR kernel, and (ii) the cache misses due to inefficient conditional instructions created by the loop.

Fig. 12a shows the results of the QUERY SELECT database kernel and in particular the compare instructions of query-06 from the TCP-H benchmark [74]. CIMX architecture achieves 4x higher performance as compared to the conventional architecture while consuming 6x less energy. The area of both architectures is shown in Fig. 12b. The improvement results from also the exploited parallelism similarly as in the case of vector OR kernel; however, as the database size does not fit within the CIM core, the speed-up is not as optimistic as the one obtained from the analytical model. With the use of non-volatile memory, the area of CIMX architecture is approximately 23% lower than that of the conventional architecture, resulting in a smaller footprint.

Overall, CIMX achieves nearly one order of magnitude improvement in delay and energy with a smaller footprint for the three simulated benchmarks. These improvements are mainly due to reduction in data movement and use of non-volatile technology.

6 DISCUSSION

In this section, we first compare the analytical model and simulation framework. Thereafter, we address the limitations of this work and suggest possible solutions.

6.1 Comparison of Analytical and Simulation Results

The comparison is based on two aspects, i.e., the accuracy and required evaluation effort.

Accuracy: The analytical and simulation results show that CIMX architecture achieves significant improvements in terms of performance/delay, energy and area. The analytical results are generally more optimistic as they assume an ideal application with a uniform profile, where consecutive kernels can be accelerated without considering the communication between them. However, typically, a program contains various kernels, and hence, the improvements depend on the accelerated parts. For example, the XOR encryption kernel achieves no speed-up in case of small problem sizes (smaller than 32KB), but the speed-up increases to more than 13x in the case of a relatively larger problem size (256MB). In addition, the improvements of the simulator are limited due to the limited parallelism of CIM core; in the analytical model, the number of parallel CIM instructions can be easily modified, while this is harder to realize in the simulator. Another difference is the configuration of the number of cores between the model and the simulation framework. In the analytical model, we assumed that the conventional architecture consists of several cores. However, the three kernels we explored with the simulation framework were run on a single core; note that a lot of efforts are needed to parallelize them. Nevertheless, the results of both the analytical model and simulation framework show similar trends and the CIMX architecture achieves nearly one

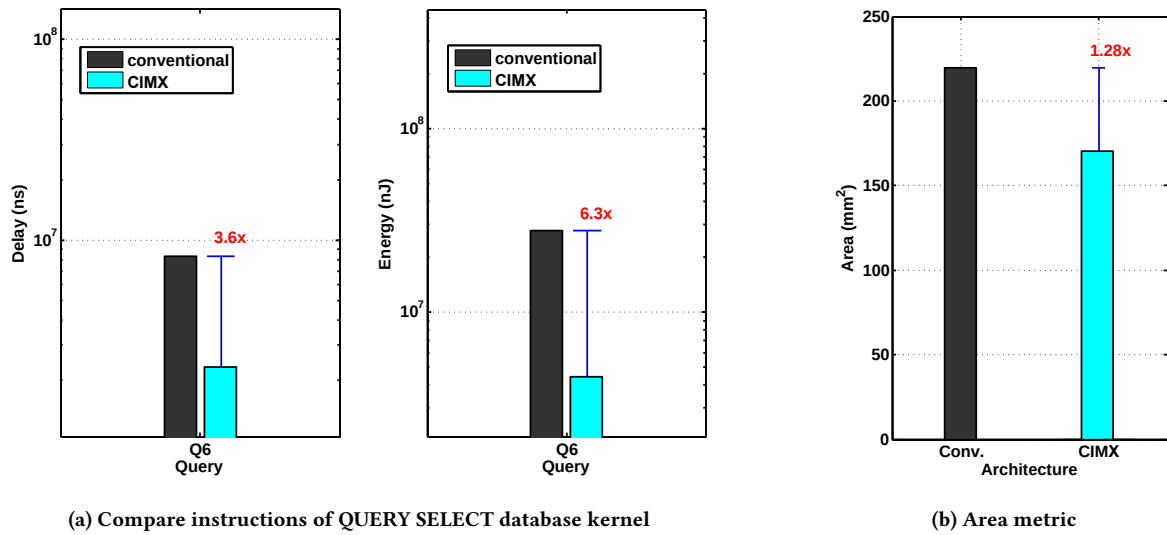


Figure 12: Delay, energy and area comparison of conventional and CIMX architecture

order of magnitude improvement in terms of speed up and energy reduction.

Evaluation effort: Even though the simulation framework provides more realistic results than the analytical model, it is time-consuming to port and execute an application on the simulation framework. First, it requires (as of now) manual insertions of pragma's and the user has to specify the CIM instructions. Second, as the simulation framework is based on a cycle-accurate simulator, even relative simple programs containing less than a hundred lines of code are transformed to traces with thousands/millions of instructions, which take hours if not days to simulate. In contrast, the analytical model provides a quick design exploration and gives a quick insight where the highest improvements can be obtained. Hence, as a lot of effort is required to run the application on the simulation platform, the analytical evaluation can be used to quickly analyze the performance estimation. This requires however accurate inputs to the model such as L1 and L2 cache miss rates. They can be retrieved for example by performing a workload characterization by for example using the Linux tool perf [76] or Intel *emon* that has been used to characterize database applications [57].

6.2 Limitations and Future Work

The proposed architecture is still an on-going research. Therefore, there are still a lot of issues to be addressed as explained next.

Mapping of complex functions on CIM core: More research is required to map complex functions on the CIM core. First, it would be ideal if arithmetic operations could be executed on CIM core. This reduces the communication between processor and main memory even further. Moreover, for database applications, operations such as *AVERAGE*, *MIN*, and *MAX* can also be performed on CIM core when these arithmetic operations are integrated; this will subsequently improve the overall performance. In case, a query has a complex operation that cannot be performed by CIM core, the results of the compare instructions have to be transferred back to the host processor to perform this complex operation; it impacts

the performance if this operation happens frequently. Therefore, it is essential to implement essential complex operations on CIM core. Third, to increase the performance even more, the instruction set can be extended with macro instructions [77] such as dot product or matrix multiplication [78]. Another approach is to integrate the arithmetic units in the CMOS layer near the memory controller, similarly as carried out in near-data computing [71]. Current prototypes have already demonstrated this approach by stacking memristors on top of a CMOS layer [79, 80]. Moreover, future work should also focus on the instruction set, as the overhead to implement and execute the in-memory instructions are often overlooked in current designs.

Memory controller and communication: The memory controller and the communication between processor and accelerator should be investigated more accurately. Optimizing the inter-communication (i.e., between processor and CIM core) and intra-communication (i.e., within CIM core) is of great importance. Similar work in the field has provided solutions to intra-communication among banks and arrays [81, 6]. However, design exploration and optimization of such components still needs further investigation and could significantly improve the overall performance and robustness [82, 77, 83, 84, 85].

CIMX architecture improvements: CIMX was designed with the intention to minimize the required number of modifications to conventional architectures. It is worth noticing that much higher performance and energy improvements can be achieved in case more radical architectures are considered, such as the one in Fig. 3d. As the CIM core stores a huge amount of data, the main DRAM might become superfluous. This will eliminate the expensive off-chip communication and enable higher performance and energy improvements. With expected technology developments, it may become feasible to allocate more and more data on the same chip. Current prototypes have demonstrated resistive memories with GBs of data [36, 86, 87]. In addition, CIMX in general and scouting logic in particular are feasible to be implemented using other memory

technologies such as SRAM and DRAM [81, 88]. Note that CIMX is based on resistive memories which generally have a low power consumption and small footprint. In order to implement CIMX using other memristor devices such as phase change memory (PCM) [89], a thorough investigation is required to estimate CIMX performance. A benefit of using PCM is that it is more robust with multi-bit cells. [90].

Application exploration and simulator automation: Although we considered three kernels, more applications have to be explored for CIMX architecture. Note that the considered CIMA architecture contains an accelerator that can only perform logical operations. Hence, it is worth exploring various applications with a high percentage of logical operations. In fact, some previous work has proposed quite a lot of applications with these characteristics such as database processing, graph processing, security encryption, and bio-sequencing [6, 81, 33, 45, 47, 91]. In this paper, we have shown potential examples such as *QUERY SELECT* kernel (database applications) and *XOR encryption* kernel (security encryption). Due to some manual processes in the simulation, it is currently not straightforward to map more potential applications. Nevertheless, exploring potential applications is essential once the simulation framework is further automated.

7 CONCLUSION

In this paper, we proposed a Computation-In-Memory Accelerator architecture that uses scouting logic to perform in-memory logical operations. Our analytical and simulation results show that CIMX architecture outperforms conventional architectures with at least one order of magnitude improvements in terms of delay and energy while consuming less chip area. These improvements can be realized for big data applications (such as those with a low data locality, large bulk bitwise operations), which perform poorly on the general purpose computers. Therefore, the proposed architecture is a viable candidate for high performance application-specific computers, embedded and/or low power system.

ACKNOWLEDGMENT

The results in this paper have been obtained in the framework of the project “Computation-in-memory architecture based on resistive devices” (MNEMOSENE), which has received funding from the European Union’s Horizon 2020 research and innovation programme under grant agreement No 780215.

The authors would also like to thank Marco Antonio Zanata Alves (UFPR), Luigi Carlo (UFRGS), and Paulo Cesar Santos (UFRGS) for many fruitful discussions and support on the simulation framework and TCP-H benchmarks.

REFERENCES

- [1] David A Patterson. 2006. Future of computer architecture. In *Berkeley EECS Annual Research Symposium (BEARS)*, College of Engineering, UC Berkeley, US.
- [2] ITRS. 2010. ITRS ERD report. <http://www.itrs.net>.
- [3] Said Hamdioui et al. 2015. Memristor based computation-in-memory architecture for data-intensive applications. In *DATE*. IEEE.
- [4] Pierre-Emmanuel Gaillardon et al. 2016. The programmable logic-in-memory (plim) computer. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 427–432.
- [5] Amir Morad et al. 2016. Resistive gp-simd processing-in-memory. *ACM Transactions on Architecture and Code Optimization (TACO)*, 12, 4, 57.
- [6] Shuangchen Li et al. 2016. Pinatubo: a processing-in-memory architecture for bulk bitwise operations in emerging non-volatile memories. In *Design Automation Conference (DAC)*, 2016 53rd ACM/EDAC/IEEE. IEEE, 1–6.
- [7] J Joshua Yang et al. 2013. Memristive devices for computing. *Nat. Nano*.
- [8] Manuel Le Gallo et al. 2017. Mixed-precision memcomputing. *arXiv preprint arXiv:1701.04279*.
- [9] Hoang Anh Du Nguyen et al. 2015. Computation-in-memory based parallel adder. In *NANOARCH*. IEEE.
- [10] Adib Haron et al. 2016. Parallel matrix multiplication on memristor-based computation-in-memory architecture. In *High Performance Computing & Simulation (HPCS)*, 2016 International Conference on. IEEE, 759–766.
- [11] Leonid Yavits et al. 2015. Resistive associative processor. *CAL*.
- [12] Garrett S Rose et al. 2012. Leveraging memristive systems in the construction of digital logic circuits. *Proceedings of the IEEE*, 100, 6, 2033–2049.
- [13] Ligang Gao et al. 2013. Programmable cmos/memristor threshold logic. *TNANO*.
- [14] E Linn et al. 2012. Beyond von neumann logic operations in passive crossbar arrays alongside memory operations. *Nanotechnology*.
- [15] Shahar Kvatinisky et al. 2014. Memristor-based material implication (imply) logic: design principles and methodologies. *TVLSI*.
- [16] Ioannis Vourkas et al. 2012. A novel design and modeling paradigm for memristor-based crossbar circuits. *Nanotechnology*, 11, 6, 1151–1159.
- [17] Shahar Kvatinisky et al. 2012. Mrl memristor ratioed logic. In *CNNA*. IEEE.
- [18] Lei Xie et al. 2016. Boolean logic gate exploration for memristor crossbar. In *DTIS*. IEEE.
- [19] Ying Wang et al. 2015. Proqram: exploiting the transparent logic resources in non-volatile memory for near data computing. In *Proceedings of the 52nd Annual Design Automation Conference*. ACM, 47.
- [20] L. Xie et al. 2017. Scouting logic: a novel memristor-based logic design for resistive computing. In *IEEE Computer Society Annual Symposium on VLSI*. IEEE, 1–6.
- [21] HA Du Nguyen et al. 2017. Memristive devices for computing: beyond cmos and beyond von neumann. In *Very Large Scale Integration (VLSI-SoC)*, 2017 IFIP/IEEE International Conference on. IEEE, 1–10.
- [22] Shahar Kvatinisky et al. 2014. Magicãã memristor-aided logic. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 61, 11, 895–899.
- [23] Julien Borghetti et al. 2010. Memristive switches enable stateful logic operations via material implication. *Nature*, 464, 7290, 873–876.
- [24] Virat Agarwal et al. 2010. Scalable graph exploration on multicore processors. In *SC*. ACM.
- [25] Jerry Chou et al. 2011. Parallel index and query for large scale data analysis. In *SC*. ACM.
- [26] Anne Siemon et al. 2014. Simulation of tao x-based complementary resistive switches by a physics-based memristive model. In *ISCAS*. IEEE.
- [27] Feng Miao et al. 2011. Anatomy of a nanoscale conduction channel reveals the mechanism of a high-performance memristor. *Advanced Materials*.
- [28] Nanoscale Integration et al. 2006. Predictive transistor model. <http://ptm.asu.edu/>.
- [29] Meng-Fan Chang et al. 2012. Endurance-aware circuit designs of nonvolatile logic and nonvolatile sram using resistive memory (memristor) device. In *Design Automation Conference (ASP-DAC)*, 2012 17th Asia and South Pacific. IEEE, 329–334.
- [30] Shruti Patil et al. 2011. Performing bitwise logic operations in cache using spintronics-based magnetic tunnel junctions. In *Proceedings of the 8th ACM International Conference on Computing Frontiers*. ACM, 33.
- [31] Jue Wang et al. 2014. Endurance-aware cache line management for non-volatile caches. *ACM Transactions on Architecture and Code Optimization (TACO)*, 11, 1, 4.
- [32] Chris H Kim et al. 2003. A forward body-biased-low-leakage sram cache: device and architecture considerations. In *Low Power Electronics and Design*, 2003. ISLPED'03. Proceedings of the 2003 International Symposium on. IEEE, 6–9.
- [33] Vivek Seshadri et al. 2015. Fast bulk bitwise and and or in dram. *IEEE Computer Architecture Letters*, 14, 2, 127–131.
- [34] Youngdon Choi et al. 2012. A 20nm 1.8 v 8gb pram with 40mb/s program bandwidth. In *Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 2012 IEEE International. IEEE, 46–48.
- [35] Richard Fackenthal et al. 2014. 19.7 a 16gb rram with 200mb/s write and 1gb/s read in 27nm technology. In *Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 2014 IEEE International. IEEE, 338–339.
- [36] Tz-yi Liu et al. 2014. A 130.7mm 2-layer 32-gb rram memory device in 24-nm technology. *IEEE Journal of Solid-State Circuits*, 49, 1, 140–153.
- [37] Chun Jason Xue et al. 2011. Emerging non-volatile memories: opportunities and challenges. In *Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, 2011 Proceedings of the 9th International Conference on. IEEE, 325–334.
- [38] Simone Raoux et al. 2008. Phase-change random access memory: a scalable technology. *IBM Journal of Research and Development*, 52, 4.5, 465–479.
- [39] Takayuki Kawahara. 2010. Scalable spin-transfer torque ram technology for normally-off computing. *IEEE Design and Test of Computers*, 1, 52–63.

- [40] Zoha Pajouhi et al. 2015. Device/circuit/architecture co-design of reliable stt-mram. In *Proceedings of the 2015 Design, Automation & Test in Europe Conference & Exhibition*. EDA Consortium, 1437–1442.
- [41] Omer Zilberberg et al. 2013. Phase-change memory: an architectural perspective. *ACM Computing Surveys (CSUR)*, 45, 3, 29.
- [42] Benjamin C Lee et al. 2009. Architecting phase change memory as a scalable dram alternative. In *ACM SIGARCH Computer Architecture News* number 3. Volume 37. ACM, 2–13.
- [43] Stamatis Vassiliadis et al. 2004. The molen polymorphic processor. *IEEE transactions on computers*, 53, 11, 1363–1375.
- [44] Razvan Nane et al. 2014. High-level synthesis in the delft workbench hardware/software co-design tool-chain. In *Embedded and Ubiquitous Computing (EUC), 2014 12th IEEE International Conference on*. IEEE, 138–145.
- [45] Junwhan Ahn et al. 2016. A scalable processing-in-memory accelerator for parallel graph processing. *ACM SIGARCH Computer Architecture News*, 43, 3, 105–117.
- [46] Amirali Boroumand et al. 2017. Lazypim: an efficient cache coherence mechanism for processing-in-memory. *IEEE Computer Architecture Letters*, 16, 1, 46–50.
- [47] Gary Benson et al. 2013. A bit-parallel, general integer-scoring sequence alignment algorithm. In *Annual Symposium on Combinatorial Pattern Matching*. Springer, 50–61.
- [48] Pawel Gepner et al. 2012. Performance evaluation of intel xeon e5-2600 family cluster using scientific and engineering benchmarks. In *Parallel Distributed and Grid Computing (PDGC), 2012 2nd IEEE International Conference on*. IEEE, 230–235.
- [49] Hadi Esmailzadeh et al. 2011. Dark silicon and the end of multicore scaling. In *Computer Architecture (ISCA), 2011 38th Annual International Symposium on*. IEEE, 365–376.
- [50] Tadaaki Yamauchi et al. 1997. A single chip multiprocessor integrated with dram. In *Workshop on Mixing Logic and DRAM, held at the 24th International Symposium on Computer Architecture*.
- [51] Allan Hartstein et al. 2006. Cache miss behavior: is itálz 2? In *Proceedings of the 3rd conference on Computing frontiers*. ACM, 313–320.
- [52] Allan Hartstein et al. 2008. On the nature of cache miss behavior: is itálz 2. *The Journal of Instruction-Level Parallelism*, 10, 1–22.
- [53] Cristina Meinhardt et al. 2013. Finfet basic cells evaluation for regular layouts. In *Circuits and Systems (LASCAS), 2013 IEEE Fourth Latin American Symposium on*. IEEE, 1–4.
- [54] Anish Mutreja et al. 2007. Cmos logic design with independent-gate finfets. In *Computer Design, 2007. ICCD 2007. 25th International Conference on*. IEEE, 560–567.
- [55] David M Fried et al. 2003. A fin-type independent-double-gate nfet. In *Device Research Conference, 2003*. IEEE, 45–46.
- [56] S Thoziyoor et al. 2008. Cacti 5.3. *HP Laboratories, Palo Alto, CA*.
- [57] Anastassia Ailamaki et al. 1999. Dbmss on a modern processor: where does time go? In *Vldb'99, Proceedings of 25th International Conference on Very Large Data Bases, September 7-10, 1999, Edinburgh, Scotland, UK* number DIAS-CONF-1999-001, 266–277.
- [58] Leonidas I Kontothanassis et al. 1994. Cache performance in vector supercomputers. In *Proceedings of the 1994 ACM/IEEE conference on Supercomputing*. IEEE Computer Society Press, 255–264.
- [59] Friman Sánchez et al. 2006. Performance analysis of sequence alignment applications. In *Workload Characterization, 2006 IEEE International Symposium on*. IEEE, 51–60.
- [60] Mark Woh et al. 2009. Anysp: anytime anywhere anyway signal processing. In *ACM SIGARCH Computer Architecture News* number 3. Volume 37. ACM, 128–139.
- [61] Chi-Keung Luk et al. 2005. Pin: building customized program analysis tools with dynamic instrumentation. In *Acm sigplan notices* number 6. Volume 40. ACM, 190–200.
- [62] I. Ashraf et al. 2014. MCPProf: Memory and Communication Profiler. Technical Report, Computer Engineering Lab CE-TR-2014-02. Delft University of Technology, Delft, Netherlands.
- [63] M. A. Z. Alves et al. 2015. Sinuca: a validated micro-architecture simulator. In *2015 IEEE 17th International Conference on High Performance Computing and Communications, 2015 IEEE 7th International Symposium on Cyberspace Safety and Security, and 2015 IEEE 12th International Conference on Embedded Software and Systems*, 605–610. doi: 10.1109/HPCC-CSS-ICSS.2015.166.
- [64] S. Li et al. 2009. Mcpat: an integrated power, area, and timing modeling framework for multicore and manycore architectures. In *2009 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 469–480.
- [65] X. Dong et al. 2012. Nvsm: a circuit-level performance, energy, and area model for emerging nonvolatile memory. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 31, 7, 994–1007. ISSN: 0278-0070.
- [66] S. J. E. Wilton et al. 1996. Cacti: an enhanced cache access and cycle time model. *IEEE Journal of Solid-State Circuits*, 31, 5, 677–688.
- [67] Aline Santana Cordeiro et al. 2017. Intrinsic-hmc: an automatic trace generator for simulations of processing-in-memory instructions. *XVIII Simpósio em Sistemas Computacionais de Alto Desempenho-WSCAD*.
- [68] Marco AZ Alves et al. 2016. Large vector extensions inside the hmc. In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2016*. IEEE, 1249–1254.
- [69] Brian R Gaeke et al. 2001. Memory-intensive benchmarks: iram vs. cache-based machines. In *Parallel and Distributed Processing Symposium., Proceedings International, IPDPS 2002, Abstracts and CD-ROM*. IEEE, 7–pp.
- [70] Jun Yang et al. 2005. Improving memory encryption performance in secure processors. *IEEE Transactions on Computers*, 54, 5, 630–640.
- [71] Paulo C Santos et al. 2016. Exploring cache size and core count tradeoffs in systems with reduced memory access latency. In *Parallel, Distributed, and Network-Based Processing (PDP), 2016 24th Euromicro International Conference on*. IEEE, 388–392.
- [72] Bruce Schneier. 2007. *Applied cryptography: protocols, algorithms, and source code in C*. John Wiley & sons.
- [73] Kimberly Keeton et al. 1998. Performance characterization of a quad pentium pro smp using oltp workloads. In *ACM SIGARCH Computer Architecture News* number 3. Volume 26. IEEE Computer Society, 15–26.
- [74] Transaction Processing Performance Council. 2015. Tpc-h, a decision support benchmark. (2015).
- [75] Paulo C Santos et al. 2017. Operand size reconfiguration for big data processing in memory. In *Proceedings of the Conference on Design, Automation & Test in Europe*. European Design and Automation Association, 710–715.
- [76] Linux. 2015. Perl linux. <https://perf.wiki.kernel.org>.
- [77] Hoang Anh Du Nguyen et al. 2017. On the implementation of computation-in-memory parallel adder. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 25, 8, 2206–2219.
- [78] Jintao Yu et al. 2018. Memristive devices for computation-in-memory. In *Design, Automation and Test in Europe DATE*.
- [79] Dmitri B Strukov et al. 2010. Hybrid cmos/memristor circuits. In *Circuits and Systems (ISCAS), Proceedings of 2010 IEEE International Symposium on*. IEEE, 1967–1970.
- [80] Kwang-Ting Tim Cheng et al. 2012. 3d cmos-memristor hybrid circuits: devices, integration, architecture, and applications. In *Proceedings of the 2012 ACM international symposium on International Symposium on Physical Design*. ACM, 33–40.
- [81] Vivek Seshadri et al. 2017. Ambit: in-memory accelerator for bulk bitwise operations using commodity dram technology. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*. ACM, 273–287.
- [82] Ioannis Vourkas et al. 2016. Alternative architectures toward reliable memristive crossbar memories. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 24, 1, 206–217.
- [83] Rotem Ben Hur et al. 2016. Memristive memory processing unit (mpu) controller for in-memory processing. In *Science of Electrical Engineering (ICSEE), IEEE International Conference on the*. IEEE, 1–5.
- [84] Peyman Pouyan et al. 2014. Reliability challenges in design of memristive memories. In *CMOS Variability (VARI), 2014 5th European Workshop on*. IEEE, 1–6.
- [85] Amirali Ghofrani et al. 2013. Towards data reliable crossbar-based memristive memories. In *Test Conference (ITC), 2013 IEEE International*. IEEE, 1–10.
- [86] Akifumi Kawahara et al. 2013. An 8 mb multi-layered cross-point reram macro with 443 mb/s write throughput. *IEEE Journal of Solid-State Circuits*, 48, 1, 178–185.
- [87] C Park et al. 2015. Systematic optimization of 1 gbit perpendicular magnetic tunnel junction arrays for 28 nm embedded stt-mram and beyond. In *Electron Devices Meeting (IEDM), 2015 IEEE International*. IEEE, 26–2.
- [88] Arun Subramanian et al. 2017. Cache automaton. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-50 '17)*. ACM, Cambridge, Massachusetts, 259–272. ISBN: 978-1-4503-4952-9. doi: 10.1145/3123939.3123986.
- [89] Stefano Ambrogio et al. 2018. Equivalent-accuracy accelerated neural-network training using analogue memory. *Nature*, 558, 7708, 60.
- [90] Can Li et al. 2018. Analogue signal and image processing with large memristor crossbars. *Nature Electronics*, 1, 1, 52.
- [91] JongWook Han et al. 1999. Optical image encryption based on xor operations. *Optical Engineering*, 38, 1, 47–55.